

线程code

1.创建线程

```
#include <pthread.h>
#include <iostream>
#include <cstring>
using namespace std;

// 总结
// 1.创建线程的本质就是：进程内的一个新执行流，执行的代码，
// 这个代码是函数，函数建立新的栈帧在共享区域。tid就是共享区的地址。
// 2.pthread_create的执行的函数 (void*)(*start_routine)(void*);
// 3.pthread_join(tid, void**), 因为返回值是void*, 所以需要二级指针进行回收。
// 两个函数都是成功返回0，失败返回error
void* start_routine(void* arg)
{
    string* info = (string*)arg;
    cout<< "传入参数: " << *info <<endl;

    info->insert(info->begin(), ' ');
    info->insert(info->begin(), ' ');
    info->insert(info->begin(), 'c');
    info->insert(info->begin(), 'i');
    info->insert(info->begin(), 'L');

    return info;
}

int main()
{
    string lic = "lichermionex";
    pthread_t tid;
    int n = pthread_create(&tid, nullptr, start_routine, &lic);
    if(n != 0)
    {
        cerr<< "pthread_create errorn : " << strerror(errno) <<endl;
        return 1;
    }

    void* ret = nullptr;
    n = pthread_join(tid, &ret);
    if(n != 0)
    {
        cerr<< "pthread_join errno : " << strerror(errno) <<endl;
        return 1;
    }

    string* retNew = (string*)ret;
    cout<< "传出参数: " << *retNew <<endl;
    return 0;
}
```

2封装线程的属性

```
#include <pthread.h>
#include <cstring>
#include <vector>
#include <iostream>
#include <unistd.h>
using namespace std;

// 总结
// 1.线程执行的函数，需要的参数可以放在对象里面进行，传参的。
// 2.vector里面有指针记得释放，然后clear,的
class threadData
{
public:
    int id;
    pthread_t tid;
    char buffer[64];
};

void* start_routine(void* arg)
{
    sleep(1);
    threadData* td = static_cast<threadData*>(arg);
    cout<< td->buffer << endl;

    return td;
}

int main()
{
    vector<threadData*> v;
    for(int i = 0; i < 5; ++i)
    {
        threadData* td = new threadData();
        td->id = i + 1;
        snprintf(td->buffer, sizeof(td->buffer), "thread %d", i + 1);

        int n = pthread_create(&td->tid, nullptr, start_routine, td);
        if(n != 0)
        {
            cerr << "pthread_create error : " << strerror(errno) << endl;
            return 1;
        }

        v.push_back(td);
    }

    for(int i = 0; i < 5; ++i)
    {
        void* ret = nullptr;
        int n = pthread_join(v[i]->tid, &ret);
```

```

    if(n != 0)
    {
        cerr << "pthread_join error : " << strerror(errno) << endl;
        return 1;
    }

    threadData* td = static_cast<threadData*>(ret);
    cout<< "线程" << td->id << "被回收了" << endl;
}

for(auto& e : v)
{
    delete e;
}

v.clear();
return 0;
}

```

位置: lesson32

3.线程返回资源

```

#include <pthread.h>
#include <cstring>
#include <vector>
#include <stdlib.h>
#include <unistd.h>
#include <iostream>
using namespace std;

// 总结
// 1.线程回收比拿到tid, 注意vector的数据放进去
// 2.注意new出来的资源和delete搭配
// 3.线程申请的资源在堆上, 不能直接在线程里面 定义临时变量拿出来的

class threadData
{
public:
    int id;
    pthread_t tid;
    char namebuffer[64];
};

class threadReturn
{
public:
    int exit_code;
    int exit_result;
};

void* start_routine(void* arg)
{

```

```

sleep(1);
threadData* td = static_cast<threadData*>(arg);
cout<< "I am " << td->namebuffer <<endl;

threadReturn* tr = new threadReturn();
tr->exit_code = 0;
tr->exit_result = 10;

return (void*)tr;
}

int main()
{
vector<threadData*> threads;
for(int i = 0; i < 10; ++i)
{
threadData* td = new threadData();
td->id = i + 1;
sprintf(td->namebuffer, sizeof td->namebuffer, "thread%d", i + 1);

int n = pthread_create(&td->tid, nullptr, start_routine, td);
if(n != 0)
{
cerr<< "pthread_create error : " << strerror(n) << endl;
return 1;
}

threads.push_back(td);
}

for(auto& iter : threads)
{
void* ret = nullptr;
int n = pthread_join(iter->tid, &ret);
if(n != 0)
{
cerr<< "pthread_join error : " << strerror(n) << endl;
return 1;
}

threadReturn* retNew = static_cast<threadReturn*>(ret);
cout<< iter->namebuffer << " success " << "exit_code:"<< retNew->exit_code
<< "exit_result:" << retNew->exit_result <<endl;
delete retNew;
}

for(auto& iter : threads)
{
delete iter;
}

threads.clear();

return 0;
}

```

